



SAFARTRAK • FLEET API

API Reference Guide

[Dashboard](#) / [Analytics](#) / [Alerts](#) / [Vehicles](#) / [Vehicle Details](#)

BASE URL <http://api.safartrak.com>
AUTH Bearer access token - see [Authentication](#), p. 3
MODULES 5
ENDPOINTS 48

Contents

Dashboard

GET	Summary
GET	Health (24h)
GET	Health (7d)
GET	Health (30d)
GET	Fleet Health
GET	Insights
GET	Actions
GET	Track Summary (today)

Analytics

GET	Summary (KPI cards)
GET	Distance Series (chart)
GET	Performance (table)
GET	Export CSV

Alerts

GET	Alerts Count
GET	Alerts (predictive)
GET	Alerts by severity
GET	Geofence Alerts (aoi, telematics)
GET	Alert Summary (4 KPI cards)
GET	Alert Types (chip grid, with counts)
GET	Alert Types (catalogue only)
GET	Alert Types (explicit window)
GET	Alert Configurations
GET	Telematics Alerts - search

Vehicles

GET	List Vehicles
GET	List Vehicles - Tracking tab
GET	List Vehicles - Maintenance tab
GET	List Vehicles - Unassigned tab
GET	List Vehicles - Fleet / type / device filters
GET	Vehicle KPI Cards
GET	Export Vehicles (CSV)
POST	Add Vehicle (whole wizard)

Vehicle Details

GET	Vehicle Details
GET	Vehicle Stats
GET	Vehicle History
PATCH	Edit - Basic Information
PATCH	Edit - Registration Details
PATCH	Edit - Driver Assignment
PATCH	Edit - GPS & Device Information
PATCH	Edit - Insurance Information
PATCH	Edit - Maintenance Information
GET	Vehicle Settings

PUT	Save Vehicle Settings
GET	List Documents
POST	Upload Document
GET	Download Document
DELETE	Delete Document
GET	Maintenance Bookings
POST	Schedule Maintenance
DELETE	Remove Vehicle (soft delete)

Before You Start

BASE URL

```
http://api.safartrak.com
```

AUTHENTICATION

All endpoints in this guide require a bearer access token, obtained via [POST /auth/login](#). Send it on every request:

```
Authorization: Bearer <ACCESS_TOKEN>
```

Access tokens expire after 15 minutes; refresh via the rotating [safartrack_refresh](#) cookie set at login.

SCOPE OF THIS GUIDE

This reference covers five modules only: **Dashboard, Analytics, Alerts, Vehicles and Vehicle Details**. For Auth, Users, Reports, Live Tracking, Area of Interest, Mobilize/Immobilize and other modules, see the full API reference.

Dashboard

Summary

Requires bearer token

GET<http://api.safartrak.com/dashboard/summary>

KPI cards. **deltas** is week-over-week: vehicle counts are diffed against a daily snapshot SafarTrak records itself (Vehk keeps no history), so they are NULL until a baseline at least 7 days old exists - null means 'not enough history yet', not zero. The *_pct figures come from Vehk's own previous-period window and read 0.0 while Vehk analytics is unpopulated.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/dashboard/summary" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Health (24h)

Requires bearer token

GET<http://api.safartrak.com/dashboard/health?range=24h>

Fleet-health gauge + time series from PMaaS /api/fleet/health-overview. range = 24h | 7d | 30d.

QUERY PARAMETERS

range	24h
-------	-----

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/dashboard/health?range=24h" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Health (7d)

Requires bearer token

GET<http://api.safartrak.com/dashboard/health?range=7d>

Same widget over 7 daily points.

QUERY PARAMETERS

range	7d
-------	----

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/dashboard/health?range=7d" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Health (30d)

Requires bearer token

GET

<http://api.safartrak.com/dashboard/health?range=30d>

Same widget over 30 daily points.

QUERY PARAMETERS

range	30d
-------	-----

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/dashboard/health?range=30d" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Fleet Health

Requires bearer token

GET

<http://api.safartrak.com/fleet/health>

Gauge score and healthy/warning/critical counts - the same health_score the Vehk portal shows.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/fleet/health" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Insights

Requires bearer token

GET

<http://api.safartrak.com/insights>

AI insights panel (PMaaS /api/ai/insights).

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/insights" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Actions

Requires bearer token

GET

<http://api.safartrak.com/actions>

Action Center: live predictive-maintenance items from PMaaS /api/alerts.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/actions" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Track Summary (today)

Requires bearer token

GET

<http://api.safartrak.com/track/summary?period=today>

Today's Summary strip: distance, fuel, trips, idle. period = today | week | month.

QUERY PARAMETERS

period	today
--------	-------

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/track/summary?period=today" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

MODULE 2 OF 5

Analytics

Fleet analytics for the Analytics screen. All four read one getAnalyticsFleetDetail call. NOTE: Vehk returns zeros for accounts where it has not populated fleet analytics - an all-zero response is upstream state, not a bug. range = 24h | 7d | 30d.

Summary (KPI cards)

Requires bearer token

GET<http://api.safartrak.com/analytics/summary?range=7d>

Distance / running / idle / halt / fleet-score, each with change_pct vs the previous window, plus best & worst vehicle.

QUERY PARAMETERS

range

7d

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/analytics/summary?range=7d" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Distance Series (chart)

Requires bearer token

GET<http://api.safartrak.com/analytics/distance-series?range=7d>

Fleet-distance chart. Vehk sends bare values with no timestamps, so labels are generated evenly across the window.

QUERY PARAMETERS

range

7d

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/analytics/distance-series?range=7d" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Performance (table)

Requires bearer token

GET<http://api.safartrak.com/analytics/performance?range=7d&sort=distance>

Per-vehicle distance, change and contribution %. Optional ?search= and ?sort=distance|change|contribution|vehicle.

QUERY PARAMETERS

range	7d
sort	distance

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/analytics/performance?range=7d&sort=distance" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Export CSV

Requires bearer token

GET

<http://api.safartrak.com/analytics/export?range=7d>

The performance table as text/csv (Content-Disposition attachment).

QUERY PARAMETERS

range	7d
-------	----

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/analytics/export?range=7d" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

MODULE 3 OF 5

Alerts

The Alerts screen mixes TWO upstream feeds that look alike and are not.

PREDICTIVE (PMaaS /api/alerts) - carries severity and a review status. Backs /alerts (no type), /alerts/count, /alerts/summary and the /alerts/live WS.

TELEMATICS (Vehk GraphQL getAlertsByType) - geofence, overspeed, ignition, harsh braking, fuel drain and so on. Backs /alerts/types and /alerts?type=. This feed has NO severity, NO review status and NO id, so those fields come back null rather than invented; the design's Status and Speed Limit columns have no upstream source. Driver and vehicle type are joined in from the cached per-vehicle lookup. Requires from/to.

Counting the chips costs one upstream call per type (Vehk exposes no per-type totals), so results are cached for 2 minutes and the fan-out capped at 20 types. Pass counts=false for the bare catalogue in a single call.

Telematics alerts still return 0 for this fleet across all types - that is upstream state, not a mapping bug.

WEBSOCKET

ws://api.safartrak.com/alerts/live?token={{accessToken}}

Alerts Count

Requires bearer token

GET`http://api.safartrak.com/alerts/count`

PMaaS alert counts: total + critical/high/medium/low + open.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts/count" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Alerts (predictive)

Requires bearer token

GET`http://api.safartrak.com/alerts`

PMaaS predictive-maintenance alerts (the portal Alerts screen). Optional filters: ?severity=critical|high|medium|low and ?status=active, plus ?limit=N.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Alerts by severity

Requires bearer token

GET

<http://api.safartrak.com/alerts?severity=medium>

Same feed filtered by normalised severity: critical | high | medium | low. Also accepts ?status= and ?limit=.

QUERY PARAMETERS

severity	medium
----------	--------

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts?severity=medium" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Geofence Alerts (aoi, telematics)

Requires bearer token

GET

<http://api.safartrak.com/alerts?type=aoi&from={{historyFrom}}&to={{historyTo}}>

Vehk GraphQL telematics alerts by type; from/to required when type is set.

QUERY PARAMETERS

type	aoi
from	2026-07-13T00:00:00Z
to	2026-07-14T00:00:00Z

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts?type=aoi&from=2026-07-13T00:00:00Z&to=2026-07-14T00:00:00Z" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Alert Summary (4 KPI cards)

Requires bearer token

GET

<http://api.safartrak.com/alerts/summary>

total, critical, high, new (not yet reviewed) and resolved. Reads the PMaaS predictive feed because it is the only alert source carrying severity and a review status.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts/summary" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Alert Types (chip grid, with counts)

Requires bearer token

GET`http://api.safartrak.com/alerts/types`

Every alert type with its count for the window (defaults to the screen's Last 7 Days). Costs one upstream call per type; cached 2 minutes.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts/types" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Alert Types (catalogue only)

Requires bearer token

GET`http://api.safartrak.com/alerts/types?counts=false`

The bare catalogue in a single upstream call - all counts are 0.

QUERY PARAMETERS

counts	false
--------	-------

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts/types?counts=false" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Alert Types (explicit window)

Requires bearer token

GET`http://api.safartrak.com/alerts/types?from={{historyFrom}}&to={{historyTo}}`

Counts over an explicit window instead of the default Last 7 Days.

QUERY PARAMETERS

from	2026-07-13T00:00:00Z
to	2026-07-14T00:00:00Z

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts/types?from=2026-07-13T00:00:00Z&to=2026-07-14T00:00:00Z" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Alert Configurations

Requires bearer token

GET`http://api.safartrak.com/alerts/configurations?type={{alertType}}`

The Configurations button: per-vehicle thresholds for one alert type. `type` is required.

QUERY PARAMETERS

<code>type</code>	<code>overspeed</code>
-------------------	------------------------

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts/configurations?type=overspeed" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Telematics Alerts - search

Requires bearer token

GET`http://api.safartrak.com/alerts?type={{alertType}}&from={{historyFrom}}&to={{historyTo}}&search={{vehicleNumber}}`

Search spans vehicle, unique id, driver and type. `severity/status/speed_limit` are null on this feed by design.

QUERY PARAMETERS

<code>type</code>	<code>overspeed</code>
<code>from</code>	<code>2026-07-13T00:00:00Z</code>
<code>to</code>	<code>2026-07-14T00:00:00Z</code>
<code>search</code>	<code>TS05UF4448</code>

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/alerts?type=overspeed&from=2026-07-13T00:00:00Z&to=2026-07-14T00:00:00Z&search=TS05UF4448" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

MODULE 4 OF 5

Vehicles

The Vehicles list screen: the table, its filter bar and tabs, the KPI cards, Import/Export and Add Vehicle. The per-vehicle panel and its edit form live in the Vehicle Details folder.

ONE CALL, TWO JOINS - rows carry the designed columns (registration, type, fleet group, driver, device status, last updated, status), which means Vehk's vehicle registry is joined to the live position feed. `counts` backs the tab badges and is computed BEFORE the tab filter, because the tabs are the filter control: collapsing them to the open tab would strand the operator.

TABS - tab accepts all | tracking | active | idle | offline | maintenance | unassigned, covering both tab sets the design draws over the same data: this screen's All / Tracking / Offline / Maintenance / Unassigned and the Vehicle Details left rail's All / Active / Maint. / Idle / Offline. Unassigned (no tracker bound) and Offline (a tracker that is not reporting) are different facts and have separate tabs.

NOT IMPLEMENTED - POST `/vehicles/import` (the "Import Data" button) and POST `/vehicles/draft` (the Add-vehicle wizard's "Save as Draft") have no endpoint yet.

List Vehicles

Requires bearer token

GET

```
http://api.safartrak.com/vehicles?search={{vehicleNumber}}&page=1&page_size=25
```

The Vehicle List table and the Vehicle Details left rail. Rows carry the designed columns - registration, type, fleet group, driver, device status, last updated and status - which means the registry is joined to the live position feed. `counts` backs the tab badges and is computed BEFORE the tab filter, because the tabs are the filter control: collapsing them to the open tab would strand the operator.

QUERY PARAMETERS

search	TS05UF4448
page	1
page_size	25

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles?search=TS05UF4448&page=1&page_size=25" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

List Vehicles - Tracking tab

Requires bearer token

GET

```
http://api.safartrak.com/vehicles?tab=tracking
```

tab accepts all | tracking | active | idle | offline | maintenance | unassigned, covering both tab sets the design draws over the same data: the list screen's All/Tracking/Offline/Maintenance/Unassigned and the Details left rail's All/Active/Maint./Idle/Offline.

QUERY PARAMETERS

tab	tracking
-----	----------

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles?tab=tracking" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

List Vehicles - Maintenance tab

Requires bearer token

GET	http://api.safartrak.com/vehicles?tab=maintenance
-----	---

Vehicles flagged in _maintenance. There is no upstream flag for a vehicle being off the road - it is set through PATCH /vehicles/{id}/maintenance or by scheduling with mark_in_maintenance=true.

QUERY PARAMETERS

tab	maintenance
-----	-------------

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles?tab=maintenance" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

List Vehicles - Unassigned tab

Requires bearer token

GET	http://api.safartrak.com/vehicles?tab=unassigned
-----	---

Vehicles with NO tracker bound. Distinct from Offline, which is a tracker that is not reporting - the design gives them separate tabs and they are different facts.

QUERY PARAMETERS

tab	unassigned
-----	------------

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles?tab=unassigned" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

List Vehicles - Fleet / type / device filters

Requires bearer token

GET	http://api.safartrak.com/vehicles?fleet={{groupName}}&type=Truck&tracking_status=connected
-----	---

The Fleet Group / Vehicle Type / Tracking status pickers on the list header. fleet matches Vehk's group name exactly; tracking_status is connected | disconnected | unassigned | no_data.

QUERY PARAMETERS

fleet	Hyderabad Zone
type	Truck
tracking_status	connected

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles?fleet=Hyderabad
Zone&type=Truck&tracking_status=connected" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Vehicle KPI Cards

Requires bearer token

GET

<http://api.safartrak.com/vehicles/summary>

One payload serves both card sets the design draws over the same fleet: the Vehicles list screen's Total / Tracking / Offline / Groups and the Vehicle Details screen's Total / Active / In Maintenance / Critical Alerts. critical_alerts is NULL rather than 0 when PMaaS is unreachable - 0 would claim "all clear", which is a different statement.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/summary" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Export Vehicles (CSV)

Requires bearer token

GET

<http://api.safartrak.com/vehicles/export?tab=all>

The "Export Data" button. Returns text/csv with the same filters as the list, un-paginated.

QUERY PARAMETERS

tab	all
-----	-----

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/export?tab=all" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Add Vehicle (whole wizard)

Requires bearer token

POST

<http://api.safartrak.com/vehicles>

The whole 6-step Add Vehicle wizard in one call. Creates a REAL vehicle on the upstream Vehk account - fill newVehicleNumber (and optionally newDeviceId) before sending; left blank it fails validation with 400 instead of creating anything.

STEP 1 vehicle_number, registration_number, vehicle_type, manufacturer, model, year, fuel_type, capacity_tons, chassis_number, engine_number, color

STEP 2 driver_name, driver_phone, license_number, driver_license_expiry, driver_emergency_contact

STEP 3 fleet_group_id, operating_region, depot, business_unit

STEP 4 tracker_id (the design's GPS Device ID), imei, sim_number, network_provider, device_model

WHAT ACTUALLY REACHES VEHK - `addVehicle` accepts seven arguments: number, type, model, uniqueDeviceId, speedLimit, capacity and the client id. Note it DOES take a capacity, which `updateVehicle` does not, so capacity_tons genuinely lands upstream on creation (and is stored too, because a later edit has no upstream write path). fleet_group_id goes through assignVehicleToGroup and sim_number through PMaaS. Everything else is stored facade-side - read `written_upstream` and `stored_safartrak` on the response.

STEP 5 IS NOT IN THIS BODY

The document uploads are files; folding four of them in would force the whole wizard to be multipart. Post them to /vehicles/{id}/documents against the id returned here. Step 6 is the success screen.

Vehicle Number and Registration Number are two inputs on step 1 and ONE field in Vehk: the number goes upstream, the registration is stored and shadows it on read.

The test script captures the new vehicle's id into uniqueId.

HEADERS

Content-Type

application/json

REQUEST BODY

```
{
  "vehicle_number": "{newVehicleNumber}",
  "registration_number": "",
  "vehicle_type": "Heavy Truck",
  "manufacturer": "Tata",
  "model": "Prima 5530",
  "year": 2024,
  "fuel_type": "Diesel",
  "capacity_tons": 12,
  "chassis_number": "",
  "engine_number": "",
  "color": "White",
  "driver_name": "",
  "driver_phone": "",
  "license_number": "",
  "driver_license_expiry": null,
  "driver_emergency_contact": "",
  "fleet_group_id": null,
  "operating_region": "",
  "depot": "",
  "business_unit": "",
  "tracker_id": "{newDeviceId}",
  "imei": "",
  "sim_number": "",
}
```

```
"network_provider": "",  
"device_model": ""  
}
```

EXAMPLE REQUEST

```
curl -X POST "http://api.safartrak.com/vehicles" \  
-H "Authorization: Bearer <ACCESS_TOKEN>" \  
-H "Content-Type: application/json" \  
-d '{  
  "vehicle_number": "{{newVehicleNumber}}",  
  "registration_number": "",  
  "vehicle_type": "Heavy Truck",  
  "manufacturer": "Tata",  
  "model": "Prima 5530",  
  "year": 2024,  
  "fuel_type": "Diesel",  
  "capacity_tons": 12,  
  "chassis_number": "",  
  "engine_number": "",  
  "color": "White",  
  "driver_name": "",  
  "driver_phone": "",  
  "license_number": "",  
  "driver_license_expiry": null,  
  "driver_emergency_contact": "",  
  "fleet_group_id": null,  
  "operating_region": "",  
  "depot": "",  
  "business_unit": "",  
  "tracker_id": "{{newDeviceId}}",  
  "imei": "",  
  "sim_number": "",  
  "network_provider": "",  
  "device_model": ""  
}'
```

Vehicle Details

The Vehicle Details screen: the panel, the five stat cards, the seven-section edit form, the settings toggles, documents, Schedule Maintenance and Remove Vehicle. The list itself is in the Vehicles folder; its KPI-card request serves this screen's header cards too.

WHAT VEHK ACTUALLY ACCEPTS - `updateVehicle` writes six fields (`vehicleNumber`, `vehicleType`, `vehicleModel`, `speedLimit`, `uniqueDeviceId`, keyed on `entityId`) and nothing else. The form designs about fifty. Manufacturer, year, VIN, colour, fuel type, every compliance expiry, the whole insurance block and the maintenance attributes have no field in either the GraphQL schema or the PMaaS OpenAPI, so they are stored in SafarTrak's `VehicleProfile` and merged onto the upstream record.

PROVENANCE - because of that split, every section write returns `written_upstream` and `stored_safartrak`, listing exactly which fields reached VEHK. Check them: without that you cannot tell an edit that will appear in the VEHK portal from one that never will, and both look like a successful save.

READABLE BUT NOT WRITABLE - capacity, engineNumber and chassisNumber are on VEHK's Vehicle type, yet `updateVehicle` cannot set them. An edited value is stored facade-side and shadows the upstream one, or the edit would appear to vanish on the next read.

IDENTITY - `{id}` is the device id (`uniqueDeviceId`), the same handle `/tracking/vehicles/{unique_id}` uses. A vehicle with no tracker has no device id at all, so VEHK's numeric `entityId` is also accepted on that segment; its facade-side rows are filed under `entity:{n}`.

SOFT DELETE - VEHK has no vehicle-delete mutation of any kind, so DELETE only hides the vehicle in SafarTrak and answers `upstream_deleted: false`. It still exists in the VEHK portal.

Vehicle Details

Requires bearer token

GET	<code>http://api.safartrak.com/vehicles/{uniqueId}</code>
-----	---

The whole detail panel, grouped by the form section that edits each block: `registry` (the design's left column), `telemetry` (its right column), `last_known_location`, `registration`, `driver`, `device`, `insurance`, `maintenance`, `settings` and `documents`. Where a field exists both upstream and in the stored overlay, the stored value wins.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/860103064914196" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Vehicle Stats

Requires bearer token

GET	<code>http://api.safartrak.com/vehicles/{uniqueId}/stats</code>
-----	---

The five stat cards above the panel. Distance comes from `getTravelHistory` and fuel from `getVehicleFuelStatistics` (which is keyed on the REGISTRATION NUMBER, not the device id); trips and engine hours from the activity feed. Average

mileage is DERIVED - the week's distance over the week's fuel - and is null when either is missing or fuel is zero, rather than an infinite km/L.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/860103064914196/stats" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Vehicle History

Requires bearer token

GET

<http://api.safartrak.com/vehicles/{{uniqueId}}/history?from={{historyFrom}}&to={{historyTo}}&snap=true>

Travel history for route playback: distance plus the point trail. from/to are ISO timestamps; snap=true snaps to roads.

QUERY PARAMETERS

from	2026-07-13T00:00:00Z
to	2026-07-14T00:00:00Z
snap	true

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/860103064914196/history?from=2026-07-13T00:00:00Z&to=2026-07-14T00:00:00Z&snap=true" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Edit - Basic Information

Requires bearer token

PATCH

<http://api.safartrak.com/vehicles/{{uniqueId}}>

Step 1. vehicle_number / vehicle_type / model reach Vehk via updateVehicle, and fleet_group_id (omitted here - add it to assign a group) goes through assignVehicleToGroup. Manufacturer, colour, fuel type, year and capacity have no upstream field and are stored facade-side. Check **written_upstream** against **stored_safartrak** on the response. An empty body is a 400, not a silent no-op.

HEADERS

Content-Type	application/json
--------------	------------------

REQUEST BODY

```
{
  "vehicle_number": "{{vehicleNumber}}",
  "vehicle_type": "Truck",
  "model": "Prima 5530",
  "manufacturer": "Tata",
```

```
{
  "color": "White",
  "fuel_type": "Diesel",
  "capacity_tons": 12,
  "year": 2024
}
```

EXAMPLE REQUEST

```
curl -X PATCH "http://api.safartrak.com/vehicles/860103064914196" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "vehicle_number": "{{vehicleNumber}}",
  "vehicle_type": "Truck",
  "model": "Prima 5530",
  "manufacturer": "Tata",
  "color": "White",
  "fuel_type": "Diesel",
  "capacity_tons": 12,
  "year": 2024
}'
```

Edit - Registration Details

Requires bearer token

PATCH
<http://api.safartrak.com/vehicles/{{uniqueId}}/registration>

Step 2. Only the registration number itself exists upstream (it IS vehicleNumber). VIN, the permit block and every expiry are absent from Vehk's schema; engine and chassis number are readable there but have no write path, so an edited value is stored here and shadows the upstream one.

HEADERS

Content-Type	application/json
--------------	------------------

REQUEST BODY

```
{
  "registration_number": "{{vehicleNumber}}",
  "vin_number": "MAT4825GH1234567",
  "engine_number": "ENG4825GHN7654321",
  "chassis_number": "CHS4625GHN1234567",
  "rc_expiry": "2027-03-31",
  "permit_type": "National",
  "permit_expiry": "2027-03-31",
  "fitness_certificate": "FC-2026-0091",
  "fitness_expiry": "2027-01-15",
  "pollution_expiry": "2026-12-01"
}
```

EXAMPLE REQUEST

```
curl -X PATCH "http://api.safartrak.com/vehicles/860103064914196/registration" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "registration_number": "{{vehicleNumber}}",
  "vin_number": "MAT4825GH1234567",
  "engine_number": "ENG4825GHN7654321",
  "chassis_number": "CHS4625GHN1234567",
  "rc_expiry": "2027-03-31",
  "permit_type": "National",
  "permit_expiry": "2027-03-31",
  "fitness_certificate": "FC-2026-0091",
  "fitness_expiry": "2027-01-15",
  "pollution_expiry": "2026-12-01"
}'
```

```
"engine_number": "ENG4825GHN7654321",
"chassis_number": "CHS4625GHN1234567",
"rc_expiry": "2027-03-31",
"permit_type": "National",
"permit_expiry": "2027-03-31",
"fitness_certificate": "FC-2026-0091",
"fitness_expiry": "2027-01-15",
"pollution_expiry": "2026-12-01"
}'
```

Edit - Driver Assignment

Requires bearer token

PATCH

<http://api.safartrak.com/vehicles/{{uniqueId}}/driver-assignment>

Step 3. Name, phone and licence go to Vehk's driver record via updateDriver - but getDriverDetailsAndGroup returns no driver id, so the driver is resolved on allPaginatedDrivers by vehicle binding first and by name second. When no driver row can be resolved, written_upstream comes back empty rather than a driver being invented. Route and shift are facade-side; Vehk's Driver has no such concept.

HEADERS

Content-Type	application/json
--------------	------------------

REQUEST BODY

```
{
  "driver_name": "Ashoke Sharma",
  "driver_phone": "9812345670",
  "license_number": "MH1420190001234",
  "current_route": "Mumbai - Pune NH48",
  "shift": "Night"
}
```

EXAMPLE REQUEST

```
curl -X PATCH "http://api.safartrak.com/vehicles/860103064914196/driver-assignment" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "driver_name": "Ashoke Sharma",
  "driver_phone": "9812345670",
  "license_number": "MH1420190001234",
  "current_route": "Mumbai - Pune NH48",
  "shift": "Night"
}'
```

Edit - GPS & Device Information

Requires bearer token

PATCH

<http://api.safartrak.com/vehicles/{{uniqueId}}/device>

Step 4. `sim_number` and `network_provider` go to PMaaS PUT `/api/devices/manage/{id}/sim` (as `sim_number` and carrier). Firmware and GPS status are reported device facts with no write path anywhere, so they are stored facade-side.

!! `tracker_id` is deliberately omitted from this body: sending it REBINDS THE VEHICLE TO A DIFFERENT DEVICE upstream, which is the only meaning Vehk gives that field.

HEADERS

Content-Type	application/json
--------------	------------------

REQUEST BODY

```
{
  "sim_number": "9876543210",
  "network_provider": "Airtel",
  "firmware": "v2.4.1"
}
```

EXAMPLE REQUEST

```
curl -X PATCH "http://api.safartrak.com/vehicles/860103064914196/device" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "sim_number": "9876543210",
  "network_provider": "Airtel",
  "firmware": "v2.4.1"
}'
```

Edit - Insurance Information

Requires bearer token

PATCH

`http://api.safartrak.com/vehicles/{{uniqueId}}/insurance`

Step 5. Entirely SafarTrak-side - Vehk has no insurance fields of any kind, so `written_upstream` is always empty here.

HEADERS

Content-Type	application/json
--------------	------------------

REQUEST BODY

```
{
  "insurance_provider": "ICICI Lombard",
  "policy_number": "POL-3311-2026",
  "coverage_type": "Comprehensive",
  "insurance_expiry": "2027-01-31",
  "claim_status": "No active claims"
}
```

EXAMPLE REQUEST

```
curl -X PATCH "http://api.safartrak.com/vehicles/860103064914196/insurance" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "insurance_provider": "ICICI Lombard",
```

```
"policy_number": "POL-3311-2026",
"coverage_type": "Comprehensive",
"insurance_expiry": "2027-01-31",
"claim_status": "No active claims"
}'
```

Edit - Maintenance Information

Requires bearer token

PATCH

<http://api.safartrak.com/vehicles/{{uniqueId}}/maintenance>

Step 6. PMaaS carries maintenance LOGS, not these per-vehicle scheduling attributes, so the whole section is stored facade-side. `in_maintenance` drives the "In Maintenance" KPI card and the list's Maintenance tab, and overrides the vehicle's live movement status - a vehicle in the workshop is not meaningfully "idle".

HEADERS

Content-Type	application/json
--------------	------------------

REQUEST BODY

```
{
  "last_service_date": "2026-06-12",
  "next_service_due": "2026-09-01",
  "odometer_km": 45682,
  "engine_hours": 1420.5,
  "maintenance_interval_km": 10000,
  "preferred_workshop": "Pune Service Depot",
  "in_maintenance": false
}
```

EXAMPLE REQUEST

```
curl -X PATCH "http://api.safartrak.com/vehicles/860103064914196/maintenance" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "last_service_date": "2026-06-12",
  "next_service_due": "2026-09-01",
  "odometer_km": 45682,
  "engine_hours": 1420.5,
  "maintenance_interval_km": 10000,
  "preferred_workshop": "Pune Service Depot",
  "in_maintenance": false
}'
```

Vehicle Settings

Requires bearer token

GET

<http://api.safartrak.com/vehicles/{{uniqueId}}/settings>

The seven toggles as currently stored. A vehicle with no profile row returns the documented defaults (all true), so a vehicle created in the Vehk portal still renders.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/860103064914196/settings" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Save Vehicle Settings

Requires bearer token

PUT

<http://api.safartrak.com/vehicles/{{uniqueId}}/settings>

PUT rather than PATCH: the screen saves the whole switch panel.

Only THREE of the seven reach Vehk - speed_alert, geofencing_alert and maintenance_alert map onto updateAlertConfiguration. live_tracking, fuel_monitoring, engine_diagnostics and trip_recording have no upstream configuration surface at all. A toggle Vehk rejects is still stored, and comes back under [stored_safartrak](#) rather than [written_upstream](#).

HEADERS

Content-Type

application/json

REQUEST BODY

```
{
  "live_tracking": true,
  "speed_alert": true,
  "fuel_monitoring": true,
  "geofencing_alert": true,
  "maintenance_alert": true,
  "engine_diagnostics": true,
  "trip_recording": true
}
```

EXAMPLE REQUEST

```
curl -X PUT "http://api.safartrak.com/vehicles/860103064914196/settings" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "live_tracking": true,
  "speed_alert": true,
  "fuel_monitoring": true,
  "geofencing_alert": true,
  "maintenance_alert": true,
  "engine_diagnostics": true,
  "trip_recording": true
}'
```

List Documents

Requires bearer token

GET

<http://api.safartrak.com/vehicles/{{uniqueId}}/documents>

RC, Insurance, Fitness, Pollution, Permit and Vehicle Image. Entirely SafarTrak-side: Vehk stores no files of any kind.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/860103064914196/documents" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Upload Document

Requires bearer token

POST

<http://api.safartrak.com/vehicles/{{uniqueId}}/documents>

multipart/form-data - pick a file in the Body tab before sending; Postman sets the boundary itself, so do not add a Content-Type header.

One current file per (vehicle, kind): re-uploading replaces the previous one, which is what choosing a file on an already-filled slot means on the design. Capped at 10 MB (VEHICLE_DOCUMENT_MAX_BYTES) and limited to pdf, png, jpg, jpeg and webp - anything else is a 400.

EXAMPLE REQUEST

```
curl -X POST "http://api.safartrak.com/vehicles/860103064914196/documents" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Download Document

Requires bearer token

GET

<http://api.safartrak.com/vehicles/{{uniqueId}}/documents/rc>

Streams the stored file for one (vehicle, kind). Last path segment is the kind: rc, insurance, fitness, pollution, permit or image.

Scoped to the caller's client - a kind with nothing stored is a 404, and Postman shows the response in the body preview or offers Save to file.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/860103064914196/documents/rc" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Delete Document

Requires bearer token

DELETE

<http://api.safartrak.com/vehicles/{{uniqueId}}/documents?kind=rc>

204 on success, 404 document_not_found when no file of that kind is stored. kind is required as a query parameter.

QUERY PARAMETERS

kind	rc
------	----

EXAMPLE REQUEST

```
curl -X DELETE "http://api.safartrak.com/vehicles/860103064914196/documents?kind=rc" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Maintenance Bookings

Requires bearer token

GET	http://api.safartrak.com/vehicles/{{uniqueId}}/maintenance-schedule
-----	---

Bookings recorded for this vehicle. **synced**: **false** marks one PMaaS did not accept - the booking is kept rather than lost.

EXAMPLE REQUEST

```
curl -X GET "http://api.safartrak.com/vehicles/860103064914196/maintenance-schedule" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

Schedule Maintenance

Requires bearer token

POST	http://api.safartrak.com/vehicles/{{uniqueId}}/maintenance-schedule
------	---

The Schedule Maintenance modal. Creates a REAL PMaaS maintenance log (POST /api/maintenance/logs).

The design's service date and service time are one instant upstream - PMaaS stores a single **scheduledDate** - so send them combined as **scheduled_at**. Estimated duration, priority and the three reminder checkboxes have NO column in that schema and are kept alongside the log id in SafarTrak; the response reports **synced** so you can tell which half landed.

mark_in_maintenance takes the vehicle off the road. It defaults to false because booking a slot and parking the vehicle are separate decisions.

HEADERS

Content-Type	application/json
--------------	------------------

REQUEST BODY

```
{
  "maintenance_type": "Preventive Service",
  "scheduled_at": "2026-08-01T09:30:00Z",
  "component": "engine",
  "duration_hours": 3,
  "priority": "high",
  "note": "Regular preventive service: engine check, oil change, filter replacement, brake inspection.",
  "notify_driver": true,
}
```

```
"notify_fleet_manager": true,
"send_email_reminder": false,
"mark_in_maintenance": false
}
```

EXAMPLE REQUEST

```
curl -X POST "http://api.safartrak.com/vehicles/860103064914196/maintenance-schedule" \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "maintenance_type": "Preventive Service",
  "scheduled_at": "2026-08-01T09:30:00Z",
  "component": "engine",
  "duration_hours": 3,
  "priority": "high",
  "note": "Regular preventive service: engine check, oil change, filter replacement, brake
inspection.",
  "notify_driver": true,
  "notify_fleet_manager": true,
  "send_email_reminder": false,
  "mark_in_maintenance": false
}'
```

Remove Vehicle (soft delete)

Requires bearer token

DELETE<http://api.safartrak.com/vehicles/{{uniqueId}}>

Vehk exposes `addVehicle` and `updateVehicle` and NO delete mutation of any kind, so this cannot remove the vehicle upstream. It is marked removed in SafarTrak, filtered out of every list and 404s on subsequent reads - and the 200 body carries `upstream_deleted: false`, because a client showing "Removed" needs to know the vehicle still exists in the Vehk portal.

There is no un-remove endpoint; clearing `removed_at` is a database operation.

EXAMPLE REQUEST

```
curl -X DELETE "http://api.safartrak.com/vehicles/860103064914196" \
-H "Authorization: Bearer <ACCESS_TOKEN>"
```